

INFRASTRUCTURE · 16 MIN READ

Model Context Protocol (MCP) Server Architecture & Implementation

Connecting LLMs to Databases, APIs, and Filesystems with Enterprise Security

Executive Architecture Summary: The authoritative guide to implementing Model Context Protocol (MCP) architectures. Covers JSON-RPC 2.0 transport, fast MCP servers, Neon Serverless Postgres integration, tool lazy loading, and security sandboxing.

1. Model Context Protocol Fundamentals

The Model Context Protocol (MCP) is an open industry standard that standardizes how AI applications connect to external tools, databases, resources, and prompt templates. MCP decouples tool implementation from the LLM core, replacing brittle ad-hoc API integrations with a robust client-host-server protocol.

- JSON-RPC 2.0 Transport: Standardized bidirectional communication over Stdio (local processes) or Server-Sent Events (SSE).
- Three Core Primitives: Tools (executable functions with side effects), Resources (read-only file/data URIs), and Prompts (reusable templates).
- Lazy Tool Resolution: Keep AI context windows clean by exposing lightweight tool catalogs and resolving full schemas on demand.
- Egress Sandboxing: Strict client authorization checks before executing potentially destructive external network requests.

2. MCP Host-Client-Server Topology

The MCP Host (e.g., Antigravity or Claude Desktop) manages the lifecycle of multiple child MCP servers, coordinating capabilities, permissions, and tool execution without exposing credentials to the model.



3. Building a Production FastMCP Server with Database Access

Implement lightweight, type-safe MCP servers with automated JSON schema validation and zero-boilerplate tool registration.

```
import { McpServer } from '@modelcontextprotocol/sdk/server/mcp.js';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
import { z } from 'zod';

const server = new McpServer({ name: 'pinnacle-lead-mcp', version: '1.0.0' });

// Register Verified Tool with Strict Input Validation
server.tool(
  'record_lead',
  { company: z.string(), email: z.string().email(), score: z.number().min(1).max(100) },
  async ({ company, email, score }) => {
    // Write to persistent database
    console.error(`[MCP] Recording lead: ${company} (${email})`);
    return { content: [{ type: 'text', text: `Lead ${company} recorded successfully at ${new Date().toISOString()}` }] };
  }
);

const transport = new StdioServerTransport();
await server.connect(transport);
```

- Serverless Neon Postgres Integration: Connect Postgres schemas with zero connection pooling bottlenecks.
- Security Isolation: Stdio servers run under dedicated process accounts with read-only filesystem fallbacks.

4. Governance, Auditing & Performance Optimization

Production MCP installations require real-time request tracing, execution timeouts, and rate limits. All tool invocations must emit structured telemetry to monitor model reliability.

- Timeout Safeguards: Enforce a strict 15-second execution limit on external database queries.
- Structured Error Feedback: Return descriptive remediation tips when queries fail to guide LLM self-correction.